EXERCICE 15 · CHANGEMENTS ENTRE LES ÉTAPES

# Migrer un écran

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1 → 2** · correction (7 fichiers)

## Étape 1 → 2 template → correction

Page détails migrée vers View / Scene / Adapter : CarDetailsScreen (view, adapter testé, scene, index) branché dans le router

**MODIFIÉ** `src/app/router.jsx` +2 −3

```
@@ -1,8 +1,7 @@
  import { createBrowserRouter } from "react-router-dom";
  import RootLayout from "./RootLayout";
  import { HomeScreen } from "@/features/cars/screens/HomeScreen";
- // TODO (exercice 15) : remplacer par import { CarDetailsScreen } from "@/features/cars/screens/CarDetailsScreen";
- import CarDetailPage from "@/pages/CarDetailPage";
+ import { CarDetailsScreen } from "@/features/cars/screens/CarDetailsScreen";
  import { FavoritesScreen } from "@/features/favorites/screens/FavoritesScreen";
  import { LoginScreen } from "@/features/auth/screens/LoginScreen";
  import { ApiDemoScreen } from "@/features/api-demo/screens/ApiDemoScreen";
@@ -19,7 +18,7 @@
      children: [
        { index: true, element: <HomeScreen /> },
        { path: "favorites", element: <FavoritesScreen />, loader: requireAuth },
-       { path: "car/:id", element: <CarDetailPage /> },
+       { path: "car/:id", element: <CarDetailsScreen /> },
        { path: "api-demo", element: <ApiDemoScreen /> },
        { path: "*", element: <NotFoundScreen /> },
      ],
```

**AJOUTÉ** `src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.adapter.js` +14 −0

```
@@ -1,0 +1,14 @@
+ import { adaptCarBase, orMissing } from "../../adapters/carAdapter";
+
+ // Réponse de l'API { Car: {...} } -> modèle de la page détails (null si la voiture est absente)
+ export const adaptCarDetails = (response, favoriteIds = []) => {
+   const dto = response?.Car;
+   if (!dto) return null;
+
+   return {
+     ...adaptCarBase(dto),
+     brand: orMissing(dto.car),
+     model: orMissing(dto.car_model),
+     isFavorite: favoriteIds.includes(dto.id),
+   };
+ };
```

**AJOUTÉ** `src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.adapter.test.js` +53 −0

```
@@ -1,0 +1,53 @@
```

```
+ import { describe, it, expect } from "vitest";
+ import { adaptCarDetails } from "./CarDetailsScreen.adapter";
+ import { MISSING_VALUE } from "../../adapters/carAdapter";
+ import { DEFAULT_COLOR } from "../../utils/colorUtils";
+
+ const dto = {
+   id: 7,
+   car: "Ford",
+   car_model: "Focus",
+   car_color: "Green",
+   car_model_year: 2012,
+   car_vin: "1FAHP3F2XCL123456",
+   price: "$12,345.00",
+   availability: false,
+ };
+
+ describe("adaptCarDetails", () => {
+   it("transforme la réponse de l'API en modèle de la page détails", () => {
+     expect(adaptCarDetails({ Car: dto })).toEqual({
+       id: 7,
+       title: "Ford Focus",
+       brand: "Ford",
+       model: "Focus",
+       colorHex: "#2ecc71",
+       colorName: "Green",
+       year: 2012,
+       vin: "1FAHP3F2XCL123456",
+       priceLabel: "$12,345.00",
+       isAvailable: false,
+       statusLabel: "Unavailable",
+       isFavorite: false,
+     });
+   });
+
+   it("indique si la voiture est en favori", () => {
+     expect(adaptCarDetails({ Car: dto }, [7]).isFavorite).toBe(true);
+   });
+
+   it("renvoie null si la voiture est absente", () => {
+     expect(adaptCarDetails(undefined)).toBeNull();
+     expect(adaptCarDetails({})).toBeNull();
+     expect(adaptCarDetails({ Car: null })).toBeNull();
+   });
+
+   it("résiste aux champs manquants", () => {
+     const car = adaptCarDetails({ Car: { id: 8 } });
+     expect(car.brand).toBe(MISSING_VALUE);
+     expect(car.model).toBe(MISSING_VALUE);
+     expect(car.vin).toBe(MISSING_VALUE);
+     expect(car.priceLabel).toBe(MISSING_VALUE);
+     expect(car.colorHex).toBe(DEFAULT_COLOR);
+   });
+ });
```

AJOUTÉ **src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.scene.jsx** +49 −0

```
@@ -1,0 +1,49 @@
+ import { Link } from "react-router-dom";
+ import { BackIcon, ColorDot, Price, StatusBadge } from "@/components/atoms";
+ import { DetailItem, FavoriteButton } from "@/components/molecules";
+
+ // Présentation pure de la fiche voiture
+ export function CarDetailsScreenScene({ car, onToggleFavorite }) {
+   return (
+     <div className="container">
```

```
+       <Link to="/" className="back-link">
+         <BackIcon />
+         Back to Catalog
+       </Link>
+
+       <div className="card detail-card">
+         <div className="detail-header">
+           <h1 className="detail-title">{car.title}</h1>
+           <FavoriteButton isActive={car.isFavorite} onToggle={onToggleFavorite} />
+         </div>
+
+         <div className="detail-grid">
+           <DetailItem label="Brand">
+             <span className="detail-value">{car.brand}</span>
+           </DetailItem>
+           <DetailItem label="Model">
+             <span className="detail-value">{car.model}</span>
+           </DetailItem>
+           <DetailItem label="Color">
+             <div className="color-detail">
+               <ColorDot color={car.colorHex} />
+               <span className="detail-value">{car.colorName}</span>
+             </div>
+           </DetailItem>
+           <DetailItem label="Year">
+             <span className="detail-value">{car.year}</span>
+           </DetailItem>
+           <DetailItem label="VIN">
+             <span className="detail-value">{car.vin}</span>
+           </DetailItem>
+           <DetailItem label="Price">
+             <Price value={car.priceLabel} className="detail-value" />
+           </DetailItem>
+           <DetailItem label="Availability">
+             <StatusBadge isAvailable={car.isAvailable} label={car.statusLabel} className="detail-value" />
+           </DetailItem>
+         </div>
+       </div>
+     </div>
+   );
+ }
```

AJOUTÉ **src/features/cars/screens/CarDetailsScreen/CarDetailsScreen.view.jsx** +28 −0

```
@@ -1,0 +1,28 @@
+ import { useParams } from "react-router-dom";
+ import { Loading } from "@/components/molecules";
+ import { useFavorites } from "@/features/favorites/hooks";
+ import { useCarDetails } from "../../hooks";
+ import { adaptCarDetails } from "./CarDetailsScreen.adapter";
+ import { CarDetailsScreenScene } from "./CarDetailsScreen.scene";
+
+ export function CarDetailsScreenView() {
+   const { id } = useParams();
+   const { data, isLoading, error } = useCarDetails(id);
+   const { favoriteIds, toggleFavorite } = useFavorites();
+
+   if (isLoading) return <Loading text="Loading car details..." />;
+
+   const car = adaptCarDetails(data, favoriteIds);
+
+   // Erreur réseau ou voiture absente : même message
+   if (error || !car) {
+     return (
```

```
+       <div className="container">
+         <p>Car not found</p>
+       </div>
+     );
+   }
+
+   // Dans Redux on stocke l'objet API brut, pas le modèle adapté
+   return <CarDetailsScreenScene car={car} onToggleFavorite={() => toggleFavorite(data.Car)}
/>;
+ }
```

AJOUTÉ **src/features/cars/screens/CarDetailsScreen/index.js** +1 −0

```
@@ -1,0 +1,1 @@
+ export { CarDetailsScreenView as CarDetailsScreen } from "./CarDetailsScreen.view";
```

SUPPRIMÉ **src/pages/CarDetailPage.jsx** +0 −98

```
@@ -1,98 +1,0 @@
- // TODO (exercice 15) : migrer cette page « à l'ancienne » vers View / Scene / Adapter.
- // Créer src/features/cars/screens/CarDetailsScreen/ avec 5 fichiers :
- //   CarDetailsScreen.view.jsx        -> appelle les hooks (useCarDetails, useFavorites), g
ère chargement / erreur / not found
- //   CarDetailsScreen.adapter.js      -> adaptCarDetails(response, favoriteIds) : fonction
pure DTO -> modèle
- //   CarDetailsScreen.adapter.test.js -> les tests Vitest de l'adapter
- //   CarDetailsScreen.scene.jsx       -> uniquement des props, assemble les composants de
@/components
- //   index.js                         -> export { CarDetailsScreenView as CarDetailsScreen
}
- // Puis brancher le router (src/app/router.jsx) sur le nouvel écran et supprimer ce fichie
r.
- // Modèle à suivre : src/features/cars/screens/HomeScreen/ (migrée en live).
- import { useQuery } from "@tanstack/react-query";
- import { useParams, Link } from "react-router-dom";
- import { useDispatch, useSelector } from "react-redux";
- import { fetchCarById } from "@/features/cars/api/carsApi";
- import { selectFavorites, toggleFavorite } from "@/features/favorites/favoritesSlice";
- import { BackIcon, HeartIcon } from "@/components/atoms";
- import { Loading } from "@/components/molecules";
- import { getColorHex } from "@/features/cars/utils/colorUtils";
-
- export default function CarDetailPage() {
-   const { id } = useParams();
-   const dispatch = useDispatch();
-   const favorites = useSelector(selectFavorites);
-
-   // L'id fait partie de la clé : chaque voiture a son propre cache
-   const { data, isLoading, error } = useQuery({
-     queryKey: ["car", id],
-     queryFn: () => fetchCarById(id),
-   });
-
-   const car = data?.Car;
-   const isFavorite = car ? favorites.some((fav) => fav.id === car.id) : false;
-
-   if (isLoading) return <Loading text="Loading car details..." />;
-
-   if (error || !car) {
-     return (
-       <div className="container">
-         <p>Car not found</p>
-       </div>
-     );
-   }
```

```
-
-   return (
-     <div className="container">
-       <Link to="/" className="back-link">
-         <BackIcon />
-         Back to Catalog
-       </Link>
-
-       <div className="card detail-card">
-         <div className="detail-header">
-           <h1 className="detail-title">
-             {car.car} {car.car_model}
-           </h1>
-           <button className={`favorite-btn ${isFavorite ? "favorite-active" : ""}`} onClick={() => dispatch(toggleFavorite(car))}>
-             <HeartIcon filled={isFavorite} />
-           </button>
-         </div>
-
-         <div className="detail-grid">
-           <div className="detail-item">
-             <span className="detail-label">Brand</span>
-             <span className="detail-value">{car.car}</span>
-           </div>
-           <div className="detail-item">
-             <span className="detail-label">Model</span>
-             <span className="detail-value">{car.car_model}</span>
-           </div>
-           <div className="detail-item">
-             <span className="detail-label">Color</span>
-             <div className="color-detail">
-               <div className="color-indicator" style={{ backgroundColor: getColorHex(car.car_color) }}></div>
-               <span className="detail-value">{car.car_color}</span>
-             </div>
-           </div>
-           <div className="detail-item">
-             <span className="detail-label">Year</span>
-             <span className="detail-value">{car.car_model_year}</span>
-           </div>
-           <div className="detail-item">
-             <span className="detail-label">VIN</span>
-             <span className="detail-value">{car.car_vin}</span>
-           </div>
-           <div className="detail-item">
-             <span className="detail-label">Price</span>
-             <span className="detail-value price">{car.price}</span>
-           </div>
-           <div className="detail-item">
-             <span className="detail-label">Availability</span>
-             <span className="detail-value ${car.availability ? "status-available" : "status-unavailable"}`}>
-               {car.availability ? "Available" : "Unavailable"}
-             </span>
-           </div>
-         </div>
-       </div>
-     </div>
-   );
- }
```